

CSV integrity check

September 7, 2025

```
[ ]: from pathlib import Path
import pandas as pd
import charset_normalizer
import requests

# =====
# Data directory
data_dir = Path("ChenShandata")
data_dir.mkdir(exist_ok=True)
print(f" Data directory: {data_dir.resolve()}")

# Download function
def download_file(url, save_path, desc=""):
    if save_path.exists():
        print(f" {desc} or {save_path.name} already exists, skipping download.")
        return
    try:
        response = requests.get(url, timeout=30)
        response.raise_for_status()
        # Use write_bytes for GeoJSON, write_text for others
        if save_path.suffix.lower() in [".geojson"]:
            save_path.write_bytes(response.content)
        else:
            save_path.write_text(response.text, encoding="utf-8")
        print(f" {desc} or {save_path.name} downloaded to {save_path.resolve()}")
    except Exception as e:
        print(f" Failed to download {desc} or {save_path.name}: {e}")
        print(" Please manually download from GitHub and place in the_
↳ 'ChenShandata' folder.")

# GitHub base URL
base_url = "https://raw.githubusercontent.com/shanchengnb/resit/main/"

# Required files
files = {
    "CSV data": {
        "url": base_url + "final_data.csv",
```

```

        "path": data_dir / "final_data.csv"
    },
    "BibTeX bibliography": {
        "url": base_url + "bio.bib",
        "path": data_dir / "bio.bib"
    },
    "CSL citation style": {
        "url": base_url + "harvard-cite-them-right.csl",
        "path": data_dir / "harvard-cite-them-right.csl"
    },
    "GeoJSON file": {
        "url": base_url + "data/neighbourhoods.geojson",
        "path": data_dir / "neighbourhoods.geojson"
    }
}

# Download files if missing
for desc, info in files.items():
    download_file(info["url"], info["path"], desc=desc)

# =====
# 3) Quick CSV check
# =====
file_path = files["CSV data"]["path"]

# Detect encoding
with open(file_path, "rb") as f:
    raw = f.read()
result = charset_normalizer.from_bytes(raw)
encoding_guess = result.best().encoding if result.best() else "Unknown"
print(f" File encoding detected: {encoding_guess}")

# Try reading CSV
try:
    df = pd.read_csv(file_path, encoding="utf-8")
    print(f" CSV successfully loaded, rows: {df.shape[0]}, columns: {df.
↪shape[1]}")
except Exception as e:
    print(f" Failed to read CSV: {e}")
    df = pd.DataFrame()

# Stop execution if DataFrame is empty
if df.empty:
    raise ValueError(
        " DataFrame is empty. Please make sure 'final_data.csv' exists in the_
↪'ChenShandata' folder, or check the GitHub download."
    )

```

```

# Check if data structure matches expectation
EXPECTED_ROWS = 60341
EXPECTED_COLS = 45

if df.shape == (EXPECTED_ROWS, EXPECTED_COLS):
    print(" Data structure is correct and matches the original GitHub file.")
else:
    print(" Warning: Data rows/columns do not match expectations!")
    print(f" Expected: {EXPECTED_ROWS} rows × {EXPECTED_COLS} columns")
    print(f" Actual: {df.shape[0]} rows × {df.shape[1]} columns")
    print(" Please ensure the CSV has not been opened/saved in Excel/WPS. It's_
↪recommended to re-download from GitHub.")

```

```

Data directory: C:\Users\Administrator\Desktop\ChenShandata
CSV data downloaded to
C:\Users\Administrator\Desktop\ChenShandata\final_data.csv
BibTeX bibliography downloaded to
C:\Users\Administrator\Desktop\ChenShandata\bio.bib
CSL citation style downloaded to
C:\Users\Administrator\Desktop\ChenShandata\harvard-cite-them-right.csl
GeoJSON file downloaded to
C:\Users\Administrator\Desktop\ChenShandata\neighbourhoods.geojson
File encoding detected: utf_8
CSV successfully loaded, rows: 60341, columns: 45
Data structure is correct and matches the original GitHub file.

```

[]: